GUÍA TÉCNICA INTEGRAL: FACTURACIÓN ELECTRÓNICA ARCA / AFIP

De Cero a la Obtención del CAE en Entorno de Homologación (Testing)

1. Introducción y Alcance del Laboratorio

Esta guía detalla el proceso completo, paso a paso, para integrar una aplicación desarrollada en Java con los Web Services de ARCA (ex AFIP) en el **entorno de homologación (testing)**. El objetivo pedagógico es que el estudiante comprenda la generación de credenciales criptográficas, la configuración institucional en el portal web y la implementación del cliente SOAP para emitir comprobantes fiscales y obtener el CAE.

Inicio

1. Ingresa a la página oficial de ARCA (producción) con tu Clave Fiscal.
2. Busca el servicio "WSASS - Autogestión Certificados Homologación" (si tu prueba requiere certificados) o dirígete al "Administrador de Relaciones de Clave Fiscal".
3. Adhiere o delega el servicio de testing específico que estás intentando probar hacia tu propio CUIT.

Resumen: Simplemente haz clic en tu enlace de testing, escribe tu CUIT y tu contraseña de AFIP de todos los días. El sistema de Autenticación Única (SUA) de AFIP reconocerá tu identidad pero operará 100% sobre las bases de datos de prueba.

2. Paso 1: Generación de Credenciales Criptográficas (Local)

Antes de configurar el portal, se deben generar la clave privada y la solicitud de certificado (CSR) utilizando OpenSSL desde la terminal (Git Bash o consola Linux/Windows).

2.1. Generar la Clave Privada RSA

```
openssl genrsa -out privada_test.key 2048
```

Esto generará un archivo llamado `privada.key`.

2.2. Generar la Solicitud de Certificado (CSR)

Ejecutar el comando especificando el CUIT de prueba (o personal habilitado en homologación):

openssl req -new -key privada.key -subj "/C=AR/O=Nombre de tu Empresa o Proyecto/CN=Nombre de tu Sistema/serialNumber=CUIT 20111111112" -out pedido.csr

```
openssl req -new -key privada_test.key -out solicitud_test.csr -subj
"/C=AR/O=TEST/SERIALNUMBER=CUIT 20291126198/CN=TEST"
```

> **Presta especial atención a esta parte del comando:** `serialNumber=CUIT 20111111112` Debes poner la palabra `CUIT`, dejar **un espacio**, y escribir el mismo CUIT con el que iniciaste sesión en AFIP.

3. Copiar y pegar en el formulario de AFIP

El comando anterior te habrá generado un archivo llamado `pedido.csr`.

Si lo abres con cualquier editor de texto (o ejecutando `cat pedido.csr` en la terminal), verás un bloque de texto que empieza y termina así:

-----BEGIN CERTIFICATE REQUEST-----

MIICvDCCAaQCAQAwdzELMAkGA1UEBhMCQVIxGzAZBgNVBAoMEk5vbWJyZSBkZSB0

(muchas líneas de texto con letras y números)

L8W2s+A/xXpQkQ==

-----END CERTIFICATE REQUEST-----

Copia **todo** ese texto (incluyendo las líneas de `BEGIN` y `END`) y pégalo exactamente en la caja del formulario de WSASS en la web de AFIP.

Una vez que envíes el formulario, AFIP validará que el CUIT de tu texto coincida con tu sesión, y te permitirá descargar el certificado final (generalmente un archivo `.crt`) para que lo configures en tu proyecto.

3. Paso 2: Configuración en el Portal Web de ARCA (Homologación)

Para operar en pruebas, el portal oficial de ingreso es https://www.afip.gob.ar/landing/default.asp (utilizando Clave Fiscal con acceso al entorno de homologación).

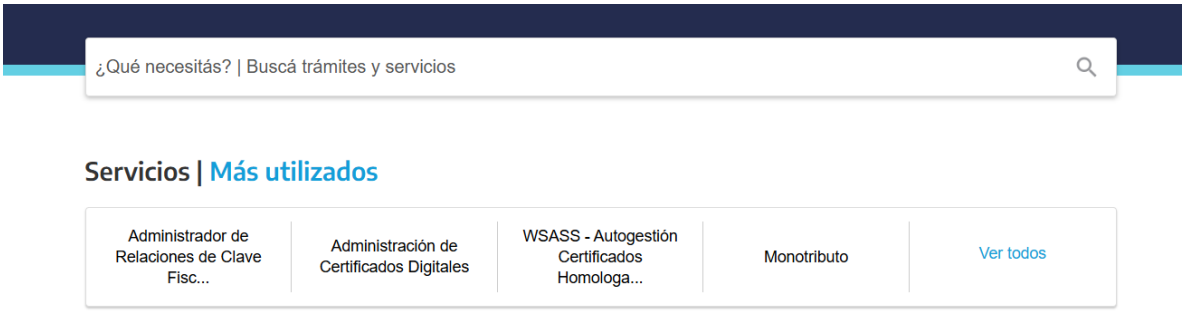


Inicia Sesion con tu clave Fiscal



Busca el servicio

WSASS Autoservicio de Acceso a WebServices (TESTING/HOMOLOGACIÓN)





3.1. Alta del Certificado Digital (Administración de Certificados)

- **1. Ingreso:** Ingresar a la web oficial de AFIP y autenticarse con Clave Fiscal.
- **2. Servicio:** Buscar y acceder al servicio Administrador de Certificados Digitales.
- **3. Crear Alias:** Seleccionar la opción Agregar Alias (ej: alias_test).
- **4. Subir CSR y Descargar:** Adjuntar el archivo solicitud_test.csr generado en el Paso 1 y guardar. Descargar el certificado resultante (ej: certificado_test.crt o .pem).

3.2. Delegación de Relaciones (Administrador de Relaciones)

- **1. Servicio:** Ingresar al servicio Administrador de Relaciones de Clave Fiscal.
- **2. Nueva Relación:** Hacer clic en Nueva Relación.
- **3. Seleccionar WSFE:** Servicio: Seleccionar AFIP -> WebServices -> Facturación Electrónica (wsfe).
- **4. Asociar Alias:** Representante: Seleccionar Computador Fiscal y elegir el alias creado (ej: alias_test). Confirmar.

4. Paso 3: Implementación del Cliente en Java (Homologación)

Para interactuar con los servicios web de ARCA/AFIP desde Java, el enfoque más limpio consiste en dividir la solución en dos componentes: el cliente **WSAA** (para obtener Token y Sign mediante firma PKCS#7/CMS con BouncyCastle) y el cliente **WSFEv1** (para enviar las peticiones SOAP a la factura electrónica).

1. Dependencias Maven

Agrega la librería BouncyCastle en tu pom.xml para realizar la firma criptográfica CMS sin depender de herramientas externas:

```
<dependencies>
  <!-- BouncyCastle para firma PKCS#7 / CMS -->
  <dependency>
    <groupId>org.bouncycastle</groupId>
    <artifactId>bcpkix-jdk18on</artifactId>
    <version>1.78</version>
  </dependency>
  <dependency>
    <groupId>org.bouncycastle</groupId>
    <artifactId>bcprov-jdk18on</artifactId>
    <version>1.78</version>
  </dependency>
</dependencies>
```

2. Obtener Token y Sign (WSAA)

Esta clase carga tu archivo certificado.pfx, genera el XML TRA de autenticación, lo firma criptográficamente y realiza la llamada SOAP a WSAA.

A continuación se presenta la estructura de código Java recomendada para estructurar la autenticación (WSAA) y la emisión de facturas (WSFE v1) en entorno de homologación.

Servicio de Autenticación WSAA (Obtención de Token y Sign)

```
import org.bouncycastle.jce.provider.BouncyCastleProvider;
import org.bouncycastle.cms.CMSSignedData;
import org.bouncycastle.cms.CMSSignedDataGenerator;
import org.bouncycastle.cms.jcajce.JcaSignerInfoGeneratorBuilder;
import org.bouncycastle.operator.jcajce.JcaContentSignerBuilder;
import org.bouncycastle.operator.jcajce.JcaDigestCalculatorProviderBuilder;
import org.bouncycastle.cert.jcajce.JcaCertStore;

import java.io.*;
```

```
import java.net.HttpURLConnection;
import java.net.URL;
import java.nio.charset.StandardCharsets;
import java.security.*;
import java.security.cert.X509Certificate;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;
import java.util.*;

public class AfipWsaaClient {

    private static final String URL_WSAA_HOMO =
"https://wsaahomo.afip.gov.ar/ws/services/LoginCms";

    public static String[] obtenerTokenYSign(String keyPath, String certPath) throws
Exception {
        if (Security.getProvider("BC") == null) {
            Security.addProvider(new BouncyCastleProvider());
        }

        PrivateKey privateKey = cargarClavePrivada(keyPath);
        X509Certificate cert = cargarCertificado(certPath);

        ZonedDateTime now = ZonedDateTime.now();
        String genTime =
now.minusMinutes(5).format(DateTimeFormatter.ISO_OFFSET_DATE_TIME);
        String expTime =
now.plusMinutes(10).format(DateTimeFormatter.ISO_OFFSET_DATE_TIME);

        String traXml = "<?xml version=\"1.0\" encoding=\"UTF-8\"?>" +
                "<loginTicketRequest version=\"1.0\">" +
                "<header>" +
                "<uniqueId>" + (System.currentTimeMillis() / 1000) + "</uniqueId>" +
                "<generationTime>" + genTime + "</generationTime>" +
                "<expirationTime>" + expTime + "</expirationTime>" +
                "</header>" +
                "<service>wsfe</service>" +
                "</loginTicketRequest>";

        CMSSignedDataGenerator gen = new CMSSignedDataGenerator();
        ContentSigner sha256Signer = new JcaContentSignerBuilder("SHA256withRSA")
                .setProvider("BC").build(privateKey);
```

```
        gen.addSignerInfoGenerator(new JcaSignerInfoGeneratorBuilder(
                new JcaDigestCalculatorProviderBuilder().setProvider("BC").build())
                .build(sha256Signer, cert));

        gen.addCertificates(new JcaCertStore(Collections.singletonList(cert)));
        CMSSignedData signedData = gen.generate(new
org.bouncycastle.cms.CMSProcessableByteArray(
                traXml.getBytes(StandardCharsets.UTF_8)), true);

        String cmsBase64 = Base64.getEncoder().encodeToString(signedData.getEncoded());

        String soapEnvelope = "<?xml version=\"1.0\" encoding=\"UTF-8\"?>" +
                "<soapenv:Envelope
xmlns:soapenv=\"http://schemas.xmlsoap.org/soap/envelope/\"
xmlns:nso=\"http://wsaa.afip.gov.ar/ws/services/LoginCms\">" +
                "<soapenv:Header/><soapenv:Body><nso:loginCms><nso:in0>" + cmsBase64 +
"</nso:in0></nso:loginCms></soapenv:Body></soapenv:Envelope>";

        String xmlResponse = enviarSoap(URL_WSAA_HOMO, soapEnvelope);

        // Limpieza y parsing básico de token y sign
        String xmlLimpio = xmlResponse.replace("&lt;", "<").replace("&gt;", ">");
        String token = xmlLimpio.substring(xmlLimpio.indexOf("<token>") + 7,
xmlLimpio.indexOf("</token>"));
        String sign = xmlLimpio.substring(xmlLimpio.indexOf("<sign>") + 6,
xmlLimpio.indexOf("</sign>"));

        return new String[]{token, sign};
    }

    private static PrivateKey cargarClavePrivada(String path) throws Exception {
        // Lógica estándar para leer archivo .key PEM
        String content = new String(Files.readAllBytes(new File(path).toPath()));
        content = content.replace("-----BEGIN PRIVATE KEY-----", "").replace("-----END
PRIVATE KEY-----", "").replaceAll("\s", "");
        byte[] keyBytes = Base64.getDecoder().decode(content);
        return KeyFactory.getInstance("RSA").generatePrivate(new
java.security.spec.PKCS8EncodedKeySpec(keyBytes));
    }

    private static X509Certificate cargarCertificado(String path) throws Exception {
        try (FileInputStream fis = new FileInputStream(path)) {
            return (X509Certificate)
java.security.cert.CertificateFactory.getInstance("X.509").generateCertificate(fis);
```

```
        }
    }

    private static String enviarSoap(String urlStr, String soapXml) throws Exception {
        URL url = new URL(urlStr);
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("POST");
        conn.setRequestProperty("Content-Type", "text/xml; charset=utf-8");
        conn.setDoOutput(true);
        try (OutputStream os = conn.getOutputStream()) {
            os.write(soapXml.getBytes(StandardCharsets.UTF_8));
        }
        try (BufferedReader br = new BufferedReader(new
InputStreamReader(conn.getInputStream(), StandardCharsets.UTF_8))) {
            StringBuilder sb = new StringBuilder();
            String line;
            while ((line = br.readLine()) != null) sb.append(line);
            return sb.toString();
        }
    }
}
```

3. Servicio de Facturación WSFE v1 (Emisión y Obtención de CAE)

Con los valores de token y sign obtenidos, ejecutas la llamada a los métodos SOAP FECompUltimoAutorizado (para saber el número siguiente) y FECAESolicitar (para solicitar el CAE).

Una vez obtenido el Token y la Sign, se invoca al WSFE v1 de Homologación (`https://wswhomo.afip.gov.ar/wsfev1/service.asmx`) utilizando bibliotecas cliente SOAP (como Apache Axis o JAX-WS).

```
// Ejemplo conceptual de llamada WSFE en Java para Homologación
// 1. Configurar credenciales de autenticación
FEAuthRequest auth = new FEAuthRequest();
auth.setToken(token);
auth.setSign(sign);
auth.setCuit(20291126198L); // CUIT de prueba

// 2. Consultar el último comprobante autorizado para calcular el siguiente número
int ptoVta = 2; // Punto de venta de prueba
int cbteTipo = 11; // Factura C
FECompUltimoAutorizadoResponse ultimoResp = serviceSoap.feCompUltimoAutorizado(auth,
```

```
ptoVta, cbteTipo);
long nuevoNro = ultimoResp.getCbteNro() + 1;

// 3. Armar la solicitud de factura (FECAESolicitar)
FECAEReq feReq = new FECAEReq();
FECAECabRequest cab = new FECAECabRequest();
cab.setCantReg(1);
cab.setPtoVta(ptoVta);
cab.setCbteTipo(cbteTipo);
feReq.setFeCabReq(cab);

ArrayFECAEDetRequest detArray = new ArrayFECAEDetRequest();
FECAEDetRequest det = new FECAEDetRequest();
det.setConcepto(1); // 1 = Productos
det.setDocTipo(99); // 99 = Consumidor Final
det.setDocNro(0L);
det.setCbteDesde(nuevoNro);
det.setCbteHasta(nuevoNro);
det.setCbteFch("20261015"); // Formato YYYYMMDD
det.setImpTotal(12100.00);
det.setImpTotConc(0.00);
det.setImpNeto(12100.00);
det.setImpOpEx(0.00);
det.setImpTrib(0.00);
det.setImpIVA(0.00); // Factura C no discrimina IVA

detArray.getFECAEDetRequest().add(det);
feReq.setFeDetReq(detArray);

// 4. Invocar el método de autorización
FECAEResponse resp = serviceSoap.feCAESolicitar(auth, feReq);

// 5. Validar resultado y extraer CAE
if ("A".equals(resp.getFeCabResp().getResultado())) {
    String cae = resp.getFeDetResp().getFECAEDetResponse().get(0).getCAE();
    String vtoCae = resp.getFeDetResp().getFECAEDetResponse().get(0).getCAEFchVto();
    System.out.println("¡Factura autorizada con éxito! CAE: " + cae + " - Vencimiento: " +
vtoCae);
} else {
    System.err.println("Rechazado por AFIP: " +
resp.getFeDetResp().getFECAEDetResponse().get(0).getObservaciones());
}
```

Ejecución del Flujo Completo

Puedes probar todo desde una clase principal Main:

```
public class Main {
    public static void main(String[] args) {
        try {
            String pfxPath = "D:\\ARCA-AFIP\\certificado.pfx";
            String pfxPassword = "ISCA132!!"; // La clave elegida al exportar el PFX
            long cuit = 20111111112L;         // Tu CUIT sin guiones
            int puntoDeVenta = 1;

            System.out.println("1. Solicitando Token y Sign a WSAA...");
            String[] auth = AfipWsaaClient.obtenerTokenYSign(pfxPath, pfxPassword, "wsfe");
            String token = auth[0];
            String sign = auth[1];
            System.out.println("Autenticado OK. Token activo.");

            System.out.println("2. Consultando último comprobante...");
            int ultimoNro = AfipWsfeClient.obtenerUltimoComprobante(token, sign, cuit, puntoDeVenta, 6);
            int nuevoNro = ultimoNro + 1;
            System.out.println("Último número registrado: " + ultimoNro + ". Siguiente a emitir: " +
nuevoNro);

            System.out.println("3. Enviando factura para CAE...");
            String respuestaCae = AfipWsfeClient.solicitarCae(token, sign, cuit, puntoDeVenta, nuevoNro,
1500.00);

            System.out.println("\n--- Respuesta WSFEv1 ---");
            System.out.println(respuestaCae);

        } catch (Exception e) {
            e.printStackTrace();
        }
```

```
    }
}
```

Alternativa para proyectos grandes:

Si prefieres no manipular cadenas XML manuales para la factura, puedes generar el código del cliente automáticamente leyendo el archivo WSDL oficial mediante la herramienta de consola de Java:

wsimport -keep -p com.afip.wsfe https://wswhomo.afip.gov.ar/wsfev1/service.asmx?WSDL

Esto creará directamente las clases Java (FECAERequest, FECAEDetRequest, etc.) para setear las propiedades como objetos.

Endpoints (Servidores de Producción)

Debes redirigir tanto WSAA como WSFE a los servidores reales de AFIP:

	Homologación (Testing)	**Producción**
WSAA	https://wsaahomo.afip.gov.ar/ws/services/LoginCms	https://wsaa.afip.gov.ar/ws/services/LoginCms
WSFE	https://wswhomo.afip.gov.ar/wsfev1/service.asmx	https://servicios1.afip.gov.ar/wsfev1/service.asmx